1. OS & ARCHITECTURE

OS Fundamentals

Resource Allocator: Manages/allocates resources.
Control Program: Controls execution & I/O.
Kernel: The one program always running.

- **Monolithic:** 1 big program (Unix, Win). Fast, crashy.
- **Microkernel:** Minimized (IPC/Mem only). Services run as user procs. Secure, stable, slow.

Hypervisors:

- **Type 1 (Bare Metal):** Replaces OS (Enterprise).
- **Type 2 (Host OS):** Runs on OS (VirtualBox).

Real-Time Systems (RTOS)

Must satisfy bounded response time; correctness relies on output *and* timeliness.

- **Focus:** Time-critical (schedulers guarantee execution upper-bound/deadline), reliable, sensor/actuator ops.
- **Response Time:** Time from input to output.

2. INTERRUPTS & POLLING

Polling vs Interrupts

Polling Cons: CPU busy-waits, wastes energy.
Interrupt System: OS is interrupt-driven (idles if none).

- Device buffer → CPU cache. Device controller fires int.
- **Flow:** Request → CPU gets int number → Look up address in **Interrupt Vector** → Execute ISR.

Interrupt Handling

- OS saves Registers & PC (preserve CPU state).
- OS determines int type; executes specific ISR code.
- OS restores context & resumes tasks.

Interrupt Response Time (T_D): Latency + Processing.

- RT req: $T_D <$ Time between interrupts.
- High-priority ints increase latency of low-priority ones.

Split Processing: ISR does minimal work (copy data, reset HW) → Queues pointer to defer main work to dedicated task (avoids missing deadlines).

System Calls

Escalates user prog to OS functions (slow/intensive).
Flow: User invokes → Lib puts syscall # in Reg → **TRAP** (switch to Kernel mode) → Handle → Return.

3. PROCESSES & THREADS

Program: Static entity. **Process:** Executing pgm.
Context Switch: Save old state, load new. Pure overhead.
Process Model: Simplifies concurrent design.

Process States

- **Created:** PCB init. Waits for minimal resources.
- **Ready:** Competes for CPU. (RTOS: must not disrupt timing of active system).
- **Running:** Actively executing on CPU.
- **Blocked:** Waiting for I/O or Sync. Not competing.
- **Terminated:** Done/Error. PCB kept for data retrieval.

Running → Ready Transitions: *Pre-emption* (Involuntary: scheduler forces out) vs *Yield* (Voluntary: process asks OS to reschedule).

Process Creation

- **fork():** Exact clone. Ret 0 to child, PID to parent. Inherits PC, Code, Data/Stack copy, Open Files.
- **exec():** Replaces memory/code with new pgm.
- **wait(status):** Parent blocks until child exits.

Threads

Unit of execution inside a process.
Shared: Heap, Data (Globals), Files, Code.
Private: Stack, Registers (PC/SP), Thread ID.

- **User Threads:** Managed by Lib. Fast. OS blind (if 1 blocks on I/O, whole process blocks). No true SMP.
- **Kernel Threads:** Managed by OS. Slower. HW-aware (True SMP). Schedulable by OS.
- **Hybrid:** User threads multiplexed on Kernel threads.

Pthreads: `pthread_t` (TID), `pthread_attr` (Attrs).

4. CONTEXT & MEMORY

Hardware Context

PC (Program Counter): Address of next instruction.
SP (Stack Pointer): Top (1st unused loc) of stack.
FP (Frame Pointer): Fixed loc in stack frame to access local vars via displacement.
Register Spilling: If GPR exhausted, save to RAM temporarily.

Calling Convention & Layout

Standard protocol for Setup/Teardown of stack frames. Stacks typically grow **upwards** (lowest addr at bottom, highest at top, incrementing SP).
Memory: Text (Instrs) → **Data** (Globals) → **Heap** (Dynamic) ↑ ↓ **Stack** (Func invocations).

5. CPU SCHEDULING

Goal: Decide who runs and for how long.
Metrics:

- **WCET (C):** Worst-Case Execution Time.
- **Deadline (D):** Max time allowed from release to completion.
- **WCRT (R):** Worst-Case Response Time (actual).

Nonpreemptive: Task holds CPU till yield/I/O.
Preemptive: CPU can be taken by OS anytime.
Interactive Sched: Focus on response/predictability. Uses timer interrupts (1-10ms) & Time Quantums.

RTOS Scheduling Algorithms

Fixed Priority: Priorities assigned manually, never change.

- **RMS (Rate Monotonic):** Optimal fixed scheme. Shorter Period = Higher Priority.

Dynamic Priority:

- **EDF (Earliest Deadline First):** Closest deadline gets highest priority.

Schedulability Analysis

Utilization (U): Sum of (C / Period) for all tasks.

- $U > 1.0$: Failed (Overloaded).
- $U \leq$ LL-Bound: Passed (Guaranteed).
- $LL < U \leq 1.0$: **Critical Instance Analysis**.

Critical Instance Analysis: Worst case scenario is all tasks release at $t = 0$.

- Step 1: Calc total WCET = t.
- Step 2: Tasks may release during t. Add new compute recursively.
- Step 3: Repeat until $t >$ Max Period (fail) or converges (pass).

Note: Can try RMS even if LL bound fails, use Critical Analysis.

Round Robin & Priority

Round Robin (RR): FIFO + Time Quantum. Fast response. Cons: Fails RTOS timing/fragile, no priority.
RR w/ Interrupts: Tasks polled, but ISR handles time-critical ints. Cons: Data processing delayed.
Priority Sched: Highest priority runs. Preemptive vs Non.

1. INTER-PROCESS COMMUNICATION (IPC)

- **Shared Memory**: Efficient but requires synchronization.
  - *POSIX Syscalls*: `shmget()` (create), `shmat()` (attach), `shmdt()` (detach), `shmctl()` (destroy).
  - *Master Code*:
```
shmid = shmget(IPC_PRIVATE, 40,
IPC_CREAT | 0600);
shm = (int*) shmat(shmid, NULL, 0);
```
- **Message Passing**: OS handles memory; portable, easier sync, but inefficient.
  - *Naming*: **Direct** (`Send(P2, Msg)`) vs. **Indirect** via Mailbox/Port (`Send(MB, Msg)`).
  - *Synchronization*: **Blocking** (Synchronous) vs. **Non-Blocking** (Asynchronous).
- **Unix Pipes**: Circular bounded byte buffer. Implicit sync (writers wait if full, readers if empty). Half/Full-duplex.
  - *Syscall*: `pipe(int fd[])`. `fd[0]` = read end, `fd[1]` = write end.
  - *Code*:
```
pipe(fd);
if(fork() > 0) {
close(fd[0]); write(fd[1], str, len);
} else {
close(fd[1]); read(fd[0], buf, len);
}
```
  - *Redirection*: `dup()`, `dup2()`.
- **Unix Signals**: Asynchronous event notification.
  - *Code*: `if (signal(SIGSEGV, myHandler) == SIG_ERR) { ... }`

2. MEMORY ABSTRACTION & ALLOCATION

- **Base + Limit Registers**: Hardware abstraction. Actual = Base + Adr. Must check Actual < Limit.
- **Contiguous Memory Allocation**:
  - **Fixed-Size Partition**: Fast, easy. Causes **Internal Fragmentation** (wasted space inside partition).
  - **Variable-Size Partition**: Flexible. Causes **External Fragmentation** (holes between partitions).
  - **Algorithms**: **First-Fit** (first hole large enough), **Best-Fit** (smallest hole large enough), **Worst-Fit** (largest hole).
- **Buddy System**: Efficient splitting/coalescing using powers of 2 (2^k).
  - Blocks B and C are buddies if their S^{th} bit is a complement and leading bits are identical.
  - Maintains an array of linked lists `A[0...K]` for free blocks of size 2^J.

3. VIRTUAL MEMORY MANAGEMENT

- **Memory Access Time**:
  $T_{access} = (1 - p) \times T_{mem} + p \times T_{page\_fault}$
  (where p = page fault probability).
- **Paging**: Splits logical memory into pages, physical memory into frames.
  - *Page Fault*: Trap to OS when accessing a non-memory resident page.
- **Page Table Structures**:
  - **Direct Paging**: Single table. Huge memory overhead (e.g., 32-bit addr, 4KiB page = 2MiB table).
  - **2-Level Paging**: Splits table into a Page Directory + smaller Page Tables. Saves space because empty directory entries mean unallocated page tables.
  - **Inverted Page Table**: One table for *all* processes, ordered by physical frame. Entry = `<PID, Page#>`. Huge memory savings, but slow translation (requires search/hashing).
- **Page Replacement Algorithms** (Triggered when physical memory is full):
  - **OPT (Optimal)**: Replace page not used for the longest future time. Unrealizable benchmark.
  - **FIFO**: Evict oldest loaded page. Suffers from **Belady's Anomaly** (more frames = *more* page faults).
  - **LRU (Least Recently Used)**: Exploits temporal locality.
    - *Implementations*: **Counter** (search for smallest time, overflow risk) or **Stack** (move referenced to top, evict bottom; hard to do in HW).
  - **CLOCK (Second-Chance)**: FIFO with a `reference bit`. If bit=0, replace. If bit=1, set to 0 (give 2nd chance) and move to next.
- **Thrashing & Working Set**:
  - **Thrashing**: Heavy I/O from constant page faults.
  - **Working Set Model**: $W(t, \Delta)$ = active pages in interval Δ. Allocate enough frames for the working set to prevent thrashing.
  - **Local vs Global Replacement**: Global allows stealing frames from other processes (can cause cascading thrashing). Local restricts a process to its own frames.

4. SYNCHRONIZATION

- **Critical Section (CS) Properties**: 1. Mutual Exclusion, 2. Progress - if nothing is in critical section, process is granted access, 3. Bounded Wait - after p_i request entry, exist an upper-bound number of times other processes can enter before p_i, 4. Independence - processes never block each other.
- **Deadlock** = no progress | **livelock** = not blocked but no progress because deadlock avoidance | **starvation** - process is blocked forever
- **Hardware Level**: `TestAndSet Reg, MemLoc`. Atomic instruction.
  - *Code*: `while(TestAndSet(Lock) == 1);`
- **Semaphore**: Integer + waiting list. No busy waiting.
  - `Wait(S)` / `P()`: If $S \leq 0$, block/sleep. Decrement S.
  - `Signal(S)` / `V()`: Increment S. Wake up one sleeping process. Never blocks.
- **Producer-Consumer (Blocking Code)**:
  - *Producer*:
```
wait(notFull); wait(mutex);
add_to_buffer;
signal(mutex); signal(notEmpty);
```
  - *Consumer*:
```
wait(notEmpty); wait(mutex);
remove_from_buffer;
signal(mutex); signal(notFull);
```
- **Readers-Writers**: Writer needs exclusive access; readers can share.
- **Pthreads**: `pthread_mutex_lock()`, `pthread_cond_wait()`, `pthread_cond_broadcast()`.

5. FILE SYSTEM

- **File Data Structure & Access**:
  - *Fixed Length Records*: Easy jump. Offset = Size × $(N - 1)$.
  - *Access Methods*: **Sequential** (read in order) vs **Random/Direct** (`read(offset)` or `seek(offset)`).
- **Directory Structures**:
  - *Single-Level* vs *Tree-Structured* (Absolute/Relative paths).
  - *DAG (Directed Acyclic Graph)*: Allows file sharing.
    - **Hard Link**: Multiple directory pointers to the same disk file. Pros: Low overhead. Cons: Deletion orphans (if owner deletes, link breaks).
    - **Symbolic Link**: Special file containing the path. Pros: Simple deletion. Cons: Disk space overhead.
- **Disk Organization**: MBR → Partitions → [OS Boot Block | Partition Details | Directory Structure | Files Info | File Data].
- **File Block Allocation**:
  - **Contiguous**: Directory stores `start` & `length`. Pros: Fast, simple. Cons: External fragmentation, file size fixed.
  - **Linked List**: Block stores data + pointer to next. Directory stores `start` & `end`. Pros: No fragmentation. Cons: Slow random access, pointer overhead.
  - **FAT (Linked List V2)**: Moves pointers to a File Allocation Table in RAM. Pros: Fast random access.
  - **Indexed**: Directory points to an *Index Block* (array of block addresses). Pros: Fast direct access. Cons: Max file size limited by index block size.
  - **Unix Inode (Combined)**: Uses direct blocks, single indirect, double indirect, and triple indirect pointers for massive files.
- **Free Space Management**:
  - **Bitmap**: 1 bit per block (0=occupied, 1=free). Fast manipulations, but takes up RAM.
  - **Linked List**: Blocks store pointers to next free block. Low memory overhead, but slow.
- **Disk I/O Scheduling**:
  - *Time* = `Seek Time` (move head to track) + `Rotational Latency` (wait for sector) + `Transfer Time`.
  - *Algorithms*: **FCFS** (First Come First Serve), **SSF** (Shortest Seek First), **SCAN** (Elevator: Bi-directional innermost ↔ outermost). Focus is on reducing *Seek Time*.

REAL-TIME SCHEDULABILITY

The iterative equation for task response time t_i is defined as:

$$t_i = \sum_{k=1}^{n \text{ tasks}} (\text{WCET of } k\text{th task}) \left\lceil \frac{t_{i-1}}{\text{period}} \right\rceil$$

$$\text{where } t_0 = \sum \text{WCET}$$

Schedulability Regions (Utilization U)

- **Not Schedulable:** Occurs when $U > 1$.
- **Necessary Condition:** $U \leq 1$.
- **May or may not be schedulable:** The region where $n(2^{1/n} - 1) < U \leq 1$.
- **Sufficient Condition:** $U \leq n(2^{1/n} - 1)$.
- **Schedulable:** Guaranteed when the utilization is within the sufficient condition range.

FUNCTION CALL CONVENTIONS

On executing function call

- **Caller:** Pass arguments with registers and/or stack.
- **Caller:** Save Return PC on stack.
- Transfer control from caller to callee.
- **Callee:** Save registers used by callee. Save old Frame Pointer (FP) and Stack Pointer (SP).
- **Callee:** Allocate space for local variables of callee on stack.
- **Callee:** Adjust SP to point to new stack top.

On returning from function call

- **Callee:** Restore saved registers, FP, and SP.
- Transfer control from callee to caller using saved PC.
- **Caller:** Continues execution in caller.

EXCEPTION/INTERRUPT HANDLER ILLUSTRATION

Process Flow

1. **Exception/Interrupt occurs:**
   - Control transfers to a handler routine automatically.
2. **Return from handler routine:**
   - Program execution resumes.
   - May behave as if nothing happened.

Handler Routine Steps

1. Save Register/CPU state.
2. Perform the handler routine.
3. Restore Register/CPU state.
4. Return from interrupt.

PROCESS CREATION USING `FORK()`

```
int main() {
    pid_t pid = fork();
    if (pid == 0) {// Child process}
    else if (pid > 0) {// Parent process}
}
```

```
int pthread_create(
    pthread_t *tidCreated,
    const pthread_attr_t *threadAttributes,
    void* (*startRoutine) (void*),
    void *argForStartRoutine
);
void pthread_exit( void* exitValue );
int pthread_join( pthread_t threadID, void **status );
```

KEY POINTERS

1. Synchronization & Interrupts

- **Disabling Interrupts:** Not a good choice for locking. Breaks quantum-based process switching (timer interrupts fail), misses important wakeup signals from ISRs, and can hang the entire system.
- **Pipes as Binary Semaphores (Locks):**
  - `fd[0]` for read, `fd[1]` for write used via **"Token"** concept.
  - **1 byte in pipe** = Lock is Available. **0 bytes** = Lock is Held.
  - **Acquire:** `read()` blocks until a byte is present, ensuring only one thread gets the token (Atomic).
  - **Release:** `write()` returns the byte, unblocking one waiting thread.

2. Process & Memory Mgmt

- **Context Switching:** Saves the state of a running process. Requires: (1) Flushing the TLB (or using ASIDs to prevent unauthorized memory access), (2) Loading the new page table, (3) Setting the PT base register.
- **Preemptive vs. Non-preemptive:** Preemptive OSs (Interactive) can forcibly interrupt processes. Non-preemptive (Real-time) cannot.
- **Memory Partitions: Fixed-Size:** Fast/simple, static, causes **Internal Frag**. **Variable-Size:** Flexible, causes **External Frag**.
- **Segmentation:** Out-of-bounds access is catchable via Segment Limit. A process's **Heap** does not create a new segment when it grows; the OS simply increases the limit of the existing heap segment.

3. Virtual Memory & Paging

- **Page Faults & Limits:** Incorrect memory access in paging is *not* inherently catchable by boundary limits unless it hits an "Invalid" (Valid Bit = 0) page.
- **Multi-level Paging:** Split memory so every page table fits *exactly* into one physical page frame.
- **Inverted Page Table:** One global table for the OS. One entry per physical frame (Index = Frame). Matches using `[PID | p | d]`.
- **Page Replacement Algorithms: OPT** (Benchmark), **FIFO** (Oldest out, Belady's Anomaly), **LRU** (Least Recently Used), **CLOCK** (Circular FIFO, skips reference bit = 1).
- **Working Set Model & Thrashing:** Used to allocate frames and prevent thrashing. Adjust the window (L or Δ) based on page fault frequency (High freq = Δ too small; low freq + low CPU = Δ too large).

4. fork(), exec(), and CoW

- **CoW Logic:** `fork()` initially shares physical frames between Parent and Child, marking them Read-Only ($W = 0$).
- **Write Attempt (Trap):** If a child modifies a RAM variable, a Memory Violation occurs. The OS catches this, copies the frame to a new physical location, and sets $W = 1$ so the child has a private copy.
- **`exec()` vs `write()`:** If the child calls `exec()`, it discards the parent's memory entirely (no CoW needed). Writing to a file descriptor (`write()`) does *not* trigger memory CoW unless the memory buffer itself is altered.

5. File Systems & I/O

- **File Descriptors (FD):** Non-negative integers serving as handles for an OS array (0 = stdin, 1 = stdout, 2 = stderr). Using `open()` gives an FD, saving the OS from expensive disk searches on every read/write.
- **Redirection (>):** To redirect standard output, the OS closes `fd=1` and points it to the new file's entry.
- **Inodes:** Store metadata and pointers to disk blocks.
- **Triply Indirect Pointers:** If a file grows into the triple-indirect range, writing just *1 byte* requires **4 disk writes** (3 index blocks + 1 data block).
- **Free Space:** Managed by Bitmaps (fast but takes RAM) or Linked Lists.

KEY EQUATIONS

1. Address Mapping & Offsets

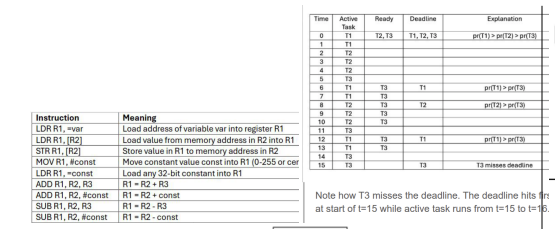
- **Base + Limit Mapping:** Actual_Physical = Base + Offset *(Must check: Offset < Limit)*
- **Buddy System Mapping:** Physical = Start_of_2^k_Block + Offset *(Must check: Offset < 2^k)*
- **File Fixed-Length Records:** Offset = Record_Size × $(N - 1)$

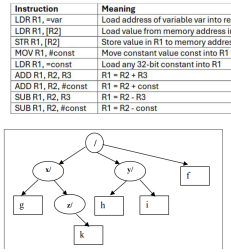
2. Memory Architecture Calculations

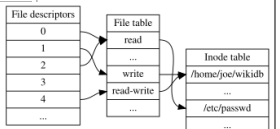
- **Number of Frames:** Physical_Address_Space ÷ Frame_Size
- **Number of Pages:** Logical_Address_Space ÷ Page_Size
- **Total Page Table Entries (PTEs):** Number_of_Pages
- **Single-Level Page Table Size:** Number_of_Pages × PTE_Size
- **Entries per Multi-Level Table:** Frame_Size ÷ Entry_Size

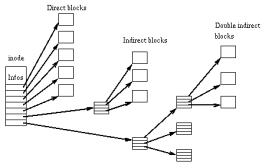
3. Performance & Access Times

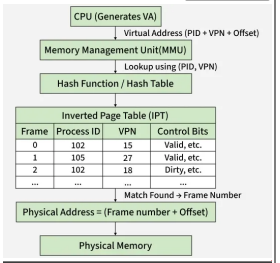
- **Effective Access Time (T_{access}):** $T_{access} = (1 - p) \times T_{mem} + p \times T_{page\_fault}$ *(where p is probability of page fault)*
- **TLB Miss Time (n-level paging):** $T_{total} = T_{TLB\_miss} + (\text{Levels} \times T_{table\_access}) + T_{mem\_access}$ *(Example: 4-level paging = 20+100+100+100+100 = 520 ns)*

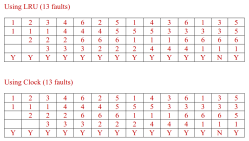


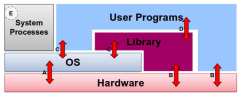


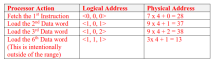


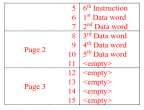


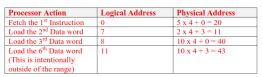


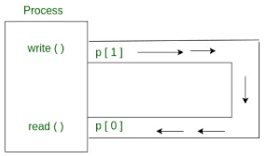


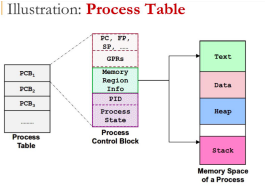




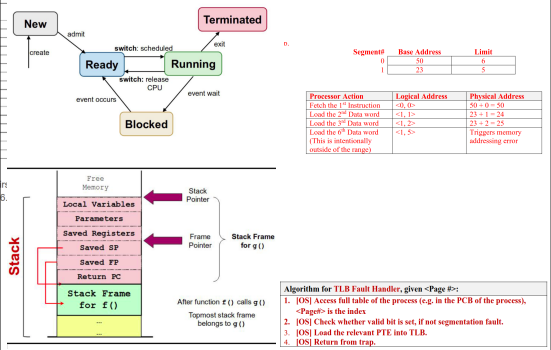








1. [OS] Global/Local replacement, Replacement algorithm applicable here
2. [OS] Write out the page to be replaced if needed.
3. [OS] Locate the page in secondary swap pages.
4. [OS] Load the page into a physical frame.
5. [OS] Update page table entries.
6. [OS] Update TLB
7. [OS] Return from Trap



Algorithm for TLB Fault Handler, given <Page #>:

1. [OS] Access full table of the process (e.g. in the PCB of the process), <Page#> is the index
2. [OS] Check whether valid bit is set, if not segmentation fault.
3. [OS] Load the relevant PTE into TLB.
4. [OS] Return from trap.

Multiples of Bytes
1000 kB = kilobyte
1024 KiB = 2^{10} bytes | MiB = 2^{20} | GiB = 2^{30}
- Arrival time: point of time at which a process enters the ready queue.
- Waiting time: amount of time spent by a process waiting in the ready queue for getting the CPU.
- Response time: amount of time after which a process gets the CPU for the first time after entering the ready queue.
- execution time or running time: amount of time required by a process for executing on CPU.
- Completion time: point of time at which a process completes its execution on the CPU and takes exit from the system.
- Turn Around Time: total amount of time spent by a process in the system.

too small: may miss pages in current locality
too big: may contain pages from different locality
110101 « 1 = 1101010
W(t,1) = start from t-1 to t inclusive